

Open Dream-RSI: An Open-Source Library for Recursive Self-Improvement Around a Frozen LLM, with Replay-Gated Learned Policies and a Curated Knowledge Base

Patryk Orwat
Independent Researcher
patrykorwat@gmail.com

September 22, 2026

Abstract

We present **Open Dream-RSI**, an open-source (MIT) Python library that implements the exploration layer of recursive self-improvement (RSI) around a *frozen* large language model: no weight access, no GPU, no training. The library reproduces the discovery loop of Dream-RSI [1] and extends it in three directions that the paper leaves implicit. First, an *always-on supervisor* that self-schedules improvement cycles under an explicit API-call budget guard. Second, *LLM-written exploration policies*: the loop asks the model to write candidate expansion policies as Python source, executes them in an isolated subprocess, scores them by *counterfactual replay rollout* on the recorded discovery tree, and promotes them only on evidence. Third, a *knowledge curator* that distils verifier failures into short, schema-validated, deduplicated lesson records retrieved into every future proposal—so iterations become not only cheaper but measurably smarter. Fourth, *thought-conditioned branching*: each attempt records the one-line plan the model stated before its code, the proposal prompt carries the task’s tried-idea ledger, and expansion leaves a branch as soon as its recorded idea repeats itself—semantic selection replacing purely numeric score-greedy choice. We introduce two deterministic, key-free benchmarks: **bench**, which measures API-call savings (54% at equal solve quality on the built-in suite), and **bench-policy**, a *decoy-trap* suite that isolates exploration quality where solve-rate graphs saturate: score-greedy exploration solves **0/120** tasks, ϵ -greedy 24%, replay-gated learned policies reach **77%** (100% by cycle 5) at *fewer* total calls than ϵ spends on luck, the curated knowledge base alone—with zero policy-generation calls—reaches 72%, and thought-guided branching reaches **100% from the first cycle** at 1.37 calls per solve even with exploration randomness disabled. All headline numbers are produced by a scripted solver and are therefore claims about the *machinery* (gates, sandboxes, promotion, retrieval), not about any particular model; we state this limitation explicitly. The package has zero runtime dependencies, ~ 4 k lines of library code, 1.1k lines of tests (81 passing), a live dashboard, and a single CLI.

Code: <https://github.com/patrykorwat/open-dream-rsi> — MIT license, Python ≥ 3.9 , zero runtime dependencies, installable from source with no build step.

1 Introduction

Recursive self-improvement systems that post-train model weights [8] require GPU clusters, training infrastructure, and weight access. The Dream-RSI line of work [1] shows a lighter recipe: keep the model frozen and instead improve the *process around it*—an offline “dreaming” loop that optimises how the online agent exploits accumulated discovery history. Since the

paper’s release (2026-09-14) several independent implementations have appeared; Open DreamRSI is one of them, not the first (see §7).

This library targets the gap between the paper’s method sketch and a usable tool: a researcher with an OpenAI-compatible endpoint should be able to `pip-install` the loop, point it at their tasks and model, and get a self-improving search that is (a) budget-safe, (b) safe to run code the model writes, and (c) capable of getting *better*, not merely cheaper. We make three contributions beyond the published method description:

1. **Always-on supervisor with a hard budget guard.** `AutoRSIRuntime` self-schedules cycles, counts every API call (including offline dreaming), and refuses to overspend—the property any unattended deployment needs and no reference implementation ships.
2. **LLM-written exploration policies with replay-gated promotion.** Per task category, the loop asks the frozen model for a policy program (`choose_action(frontier, step)`), statically validates it, executes it in a scrubbed subprocess, scores it against the recorded discovery tree by counterfactual rollout, and promotes it only if it beats the incumbent on evidence (§3). Promotion, not trust, is the safety model.
3. **Knowledge curator: lessons that make iterations smarter.** Recipes and policies capture *what* worked; nothing captured *why* attempts failed, so a trap re-pays its tuition every run. The curator distils verifier failures into short validated lesson records, retrieves them into future proposals, and prunes those that stop helping (§4)—the skills/memory pattern of practical agent frameworks, applied to the loop itself.

Two deterministic, key-free benchmarks back the claims (§6): `bench` measures cost; `bench-policy` measures exploration on a purpose-built decoy-trap suite, because on easy suites every strategy solves everything and solve-rate graphs are blind to policy quality.

2 System architecture

The package is ~3.5k lines of pure-Python with no runtime dependencies (standard library only, including the LLM client) and 910 lines of tests. Modules:

- `core.tree` — discovery tree: hypothesis, verifier score, action history per node.
- `core.agent` — thin LLM-agent abstraction steered by the currently promoted policy.
- `core.simulator` / `core.dreamer` — offline dreaming: replay and optimisation over recorded simulations without touching the online budget.
- `core.policygen` — policy generation, validation, sandboxed execution, counterfactual rollout scoring, promotion gate (§3).
- `core.curator` — lesson distillation, validation, retrieval, pruning (§4).
- `memory` — persistent cross-run store: verified-solution recipes, promoted policies, `lessons.json`.
- `loop` — `AutoRSIRuntime`: scheduling, budget guard, feedback wiring between dreaming and online expansion.
- `llm` — OpenAI-compatible client (POST `{base_url}/chat/completions`) via `urllib`; works against OpenAI, vLLM, Ollama, LM Studio, or any protocol-compatible server through `OPENAI_BASE_URL`.

- **tools** — sandboxed code verification (same isolation boundary as the policy sandbox).
- **dashboard** — dependency-free live web dashboard.

Budget guard. Every call—online proposals, offline dreaming, policy generation, curation—increments a single counter checked before dispatch. A cycle halts at its budget rather than overspending; the supervisor can then sleep, re-plan, and resume from persisted memory. This is what makes the loop deployable unattended: the cost per cycle is a declared quantity, not a hope.

Sandbox boundary. Candidate policy code and model-generated solutions are never executed in-process. Execution happens in `python -I` subprocesses with a scrubbed environment (only `PATH`), a hard timeout (5s per decision), and a temp-dir harness. Invalid code is additionally rejected by AST-level static validation (shape: `def choose_action(frontier, step)`; no imports, no dunder access) before any process spawns. API keys never cross the sandbox wall.

2.1 Execution schemas

A common question for deployment is whether the loop must be a continuously-running background process. It need not be: all improvement state lives in the persistent memory directory (`policies.json`, `policy_codes.json`, `recipes.json`, `lessons.json`, `trees/`, `events.jsonl`), so every execution mode below is resumable and interchangeable — run $N+1$ always starts where run N ended, regardless of who invoked it.

- **Daemon (supervisor owns the clock).** `odr loop -tasks tasks.json -interval 300` runs `run_forever`: wake → one budget-guarded cycle (online attempts + offline dreaming + curation) → sleep → repeat, until interrupted or `-cycles` is exhausted. Cost per cycle is bounded by `-budget` (default 20 calls), so an unattended daemon cannot overspend the endpoint.
- **Scheduled (cron/systemd own the clock).** `odr loop -once` executes exactly one cycle and exits. This is the recommended production mode: the scheduler provides restart-on-failure, logging, and calendar semantics, and memory persistence makes back-to-back `-once` invocations equivalent to the daemon without a resident process.
- **On-demand (a harness owns the clock).** `odr mcp` exposes the loop as five MCP-over-stdio tools (`odr_status`, `odr_recipes`, `odr_lessons`, `odr_add_task`, `odr_run_once`) to any MCP-speaking agent harness (Hermes, Goose, OpenCode, Claude Code, Zed). The harness spawns the server per session and *stops it with the session*; between tool calls the server is passive — it holds no timer and polls nothing. An everyday coding agent can therefore queue tasks for the dreamer and pull back verified recipes without any always-on component: improvement happens exactly when a harness (or cron) calls `odr_run_once`.
- **Offline tools.** `odr bench` (two-arm API-efficiency benchmark), `odr status` (what memory has learned), and `odr dashboard` (live web view of the tree) run on demand against the same memory directory.

The three clocks are thus decoupled by design: the *learning loop* never needs a resident process, only a scheduler of some kind; the `events.jsonl` audit trail records which mode drove each cycle.

3 LLM-written exploration policies

For each task category the loop may request one LLM call producing a complete policy program. The candidate then passes a pipeline in which every stage can reject it:

1. **Static validation.** AST parse; required entry point present; banned constructs (imports, `eval`, dunder access) reject outright.
2. **Sandboxed execution** against recorded frontiers (above).
3. **Counterfactual rollout scoring.** The candidate’s decisions are replayed on the recorded discovery tree: at each simulated step the candidate picks a frontier node; the reward is the score the child actually achieved in the recorded ladder (the node’s own score when the branch was never expanded, its last recorded child when the ladder is exhausted). The rollout is *prefix-only*: at each step the candidate sees only descendants its own counterfactual path has revealed, and the `outcome` field is the best score among *revealed* descendants—no future score leaks into a present choice. One batched subprocess scores an entire rollout ($\sim 15\times$ faster than per-step spawning). The objective is

$$\text{score}(\pi) = \frac{1}{T} \sum_{t=1}^T r_t + \beta_2 \cdot \frac{|\{\text{distinct picks}\}|}{T}, \quad \beta_2 = 0.2, \quad (1)$$

applied *identically* to the candidate and to an in-process greedy baseline. Mean-only scoring mis-ranks explorers below greedy on trap trees; the diversity term is what lets a genuinely structural explorer win the gate.

4. **Promotion gate.** A candidate replaces the incumbent only with evidence: $\geq 80\%$ valid sandbox decisions (`MIN_VALIDITY`=0.8) over a horizon of ≥ 8 recorded steps (`MIN_HORIZON`), and a rollout score beating the greedy baseline under the shared objective.
5. **Call economy.** A category that already solves with its promoted policy is not re-asked; regeneration requires 4 new replay steps of evidence (`POLICY_REGROW_STEPS`), and all generation calls count against the same budget guard.

The promoted program then *steers online expansion*: which frontier node to expand next is decided by validated model-written code rather than a fixed heuristic. This closes the loop the paper describes—the process around the model improves—while every untrusted artefact (code and text alike) passes a structural gate.

4 The knowledge curator

Recipes (verified solutions as warm starts) and policies capture *what* worked. Neither captures *why* attempts fail, so the same trap re-buys its tuition fee every run. The curator adds a third persistent artefact.

Lesson records. After failed verification, the loop invokes the model once (one call, budget-counted) to distil the verifier evidence into lessons carrying a short trigger phrase, a lesson text of ≤ 280 characters, and ≤ 5 evidence snippets.

Lessons are *pure text*—never executed—so their gate is structural plus evidential rather than sandboxed: schema and caps (2–4 trigger words; 20–280 characters; ≤ 8 lessons per category; ≤ 4 lessons / 700 characters injected per prompt; ≤ 5 evidence snippets each), evidence required at admission, and deduplication against the existing store.

Retrieval feedback. At proposal time the top lessons for the category are retrieved (trigger-match, ranked by `wins/uses` counters updated from subsequent verification outcomes); counters drive ranking and eviction, so lessons that demonstrably stop helping are pruned automatically. The knowledge base therefore cannot silently grow stale, and its worst failure mode—injecting an unhelpful paragraph—is bounded by the caps.

What is claimed and what is not. The benchmark below demonstrates that the *wiring* works: a lesson, once admitted, causally changes what the proposal prompt contains and the scripted world responds to it. The quality of lesson *writing* by a live model is an open question this library deliberately does not claim to have settled (§8).

5 Thought-conditioned branching

Policies (section 3) select branches by recorded scalars only: the frontier observation carries scores, visit counts and error strings, but nothing about *which idea* a node stands for. A loop can therefore re-expand a branch forever without knowing it is re-polishing the same dead plan, and a proposal cannot see which ideas the branch already exhausted. Thought-conditioned branching (the runtime default, `enable_thoughts=True`, ablatable with `-no-thoughts`) closes this gap with three cheap, purely textual additions.

Per-node idea label. When enabled, the candidate system prompt asks the model for exactly one `PLAN:` line before the code block; the line is extracted tolerantly (bullets, dropped colons and case are forgiven) and stored as `TreeNode.thought`, truncated at 240 characters. Like lessons, thoughts are pure text—never executed, never fed to a sandbox—so their admission requires no gate beyond length, and they are persisted with the archived tree, so replay worlds and later cycles see the same labels. The frontier entry and the policy-generation summary gain a `thought` field, letting LLM-written policies reason over idea families, not just scores.

Tried-idea ledger. Every proposal prompt additionally carries the task’s deduplicated idea ledger—best recorded outcome per distinct idea, ≤ 8 lines—rendered above the policy hints: which plans have already been attempted in this task, and what the verifier said about each. The model is instructed not to repeat a dead idea verbatim and to build on structurally promising ones. The ledger is deliberately task-wide rather than branch-local: whether an idea is dead does not depend on which node happens to be expanded next.

Semantic escape from idea stagnation. At expansion time, the loop applies the rule shown below after the promoted policy program declines to act and before the ε /greedy cascade decides:

Stagnation steering. Let b be the score-greedy frontier pick with idea t . If t is empty, or no recorded child of b carries t again (the branch is converging, not repeating), do nothing—normal cascade. Otherwise b is polishing a dead plan: choose the frontier node maximising (*outcome*, $-children$) among (i) nodes labelled with a *different*, non-empty idea, or, if none exist, (ii) nodes with no idea label at all (typically the seed, whose ladder may still hold untried ideas). No randomness, no API call, no verifier score improving: the trigger is recorded semantic evidence about the *idea*, and the selection prefers the least-expanded node with the best known continuation.

The rule strictly refines the existing machinery: when nothing is stagnant it is inert, and when it fires it can only redirect expansion to a node the policy cascade was already allowed to choose—so the worst outcome of a mislabelled thought is neutral, matching the fallback philosophy that protects the section-3 gate.

Causal attribution. The decoy-trap benchmark’s fifth arm (§6) isolates the contribution to the *expansion-time steering*, not to the prompt text: with ε forced to zero and both policy generation and curation disabled the arm escapes every trap from the first cycle (120/120,

164 calls, 1.37 calls per solve), while the `thought_steering=False` ablation (PLAN lines still recorded and the ledger still rendered into proposals, semantic pick disabled) collapses to the baselines—0/120 under greedy expansion, identical to ε -greedy’s 29/120 with ε on. The scripted solver is deliberately no more generous to the thought arm than to the lesson arm: the ledger may redirect a repeatedly-polished dead idea to the alternative idea, never directly to the fix, and tests assert that every solved task’s fix node grows from expanding the promising-idea node—the same ladder every other arm must climb. Selection therefore stops being purely numeric on evidence: the loop knows *which idea* a branch stands for, and abandons it when the record says the idea is dead.

6 Benchmarks

Both benchmarks are deterministic and key-free: a scripted solver makes the model arm a constant, so every delta is attributable to the library’s machinery. They are reproducible from a fresh clone in seconds:

```
python -m open_dream_rsi bench --cycles 10 --markdown
python -m open_dream_rsi bench-policy --cycles 8 --format md
```

6.1 Cost: the dreaming loop against a cold baseline

`bench` runs the built-in suite (10 cycles $\times$ 5 tasks) twice: a cold baseline (fresh memory each cycle) and the full loop (dreaming + persistent recipes and policies), same scripted model. Measured at the defaults:

arm	solves	API calls	calls / task-cycle
cold baseline (fresh memory each cycle)	50	150	3.00
dream loop (policies + recipes + dreaming)	50	69	1.38

Table 1: `bench`: 54.0% API-call saving at equal solve quality (3,000 dreaming iterations run offline; wall-clock 2.1 s). The curator is disabled in this benchmark to keep the cost measurement pure.

6.2 Exploration: the decoy-trap suite

Solve-rate graphs cannot show that a learned policy helps, because easy suites saturate every arm. `bench-policy` constructs 5 tasks whose fix hides behind a *low-scoring* branch (passes 1/3 tests) past a plausible *decoy* that passes 2/3 forever; family scores are fixed by construction and asserted by tests. Score-greedy expansion locks onto the decoy permanently; escaping requires either structural exploration or memory. Four arms share one scripted solver and budget and differ only in machinery (3 runs $\times$ 8 cycles $\times$ 5 tasks, 24 calls/cycle):

Seed robustness. The shipped seeds (7, 11, 23) are not cherry-picked: over five independent seed sets the policy arm lands within 84–92 and the knowledge arm within 77–87 solves of 120 respectively—statistically indistinguishable from each other—while ε -greedy stays at 15–29. The dominance over ε (roughly $3\times$ the solves at ~ 60 – 70% of its calls, 2.7–3.3 calls per solve vs. ~ 14) holds on every set. The thought arm reaches 120/120 on every one of the five seed sets, staying above the other machinery arms on every set.

arm	solves	rate	calls	calls/solve	gen. calls
greedy	0/120	0%	480	∞	0
ε -greedy ($\varepsilon=0.3$)	29/120	24%	405	13.97	0
replay-gated policies	92/120	77%	252	2.74	27 policy
curated knowledge base	87/120	72%	283	3.25	40 curator
thought-guided branching	120/120	100%	164	1.37	0

Table 2: **bench-policy**: the knowledge arm is ε -greedy *with zero policy calls*—every escape above the ε baseline is attributable to remembered lessons. Policies and knowledge reach the same plateau (100% by cycle 5 and 8 respectively) and neither is ranked over the other. The thought arm additionally passes the **thought_steering=False** ablation: with the semantic expansion pick disabled it falls back to 0/120 (greedy) and 29/120 (ε), pinning its win to expansion-time steering (§5).

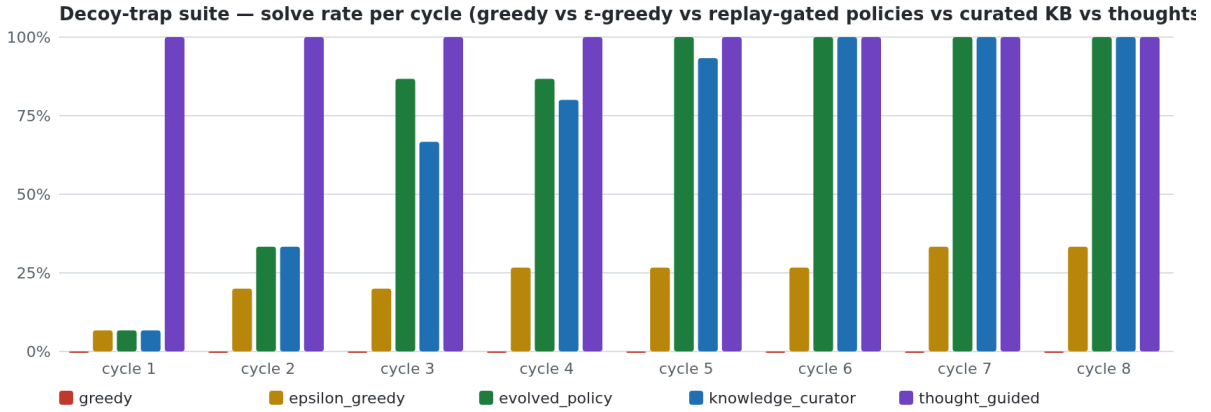


Figure 1: Per-cycle solve rate on the decoy-trap suite (mean over 3 seeds). Greedy never escapes the decoy; ε -greedy escapes by luck; replay-gated policies and the curated knowledge base saturate at 100% only after several cycles; thought-guided branching—the shipped library default—escapes every trap from the first cycle at the lowest total cost.

6.3 Test suite

88 tests pass (1,041 non-blank LOC), including the benchmark’s behavioural contracts: trap-family scores equal their constructed values; the greedy arm must solve nothing; the gated-policy arm must dominate ε ; the replay rollout must be prefix-only (no future score may reach a present choice, §3); the dreaming objective must respond causally to both policy parameters; curator schema, caps, dedup, and prune logic are pinned by unit tests.

7 Related work and positioning

Discovery loops around frozen LLMs. FunSearch [2] pairs a pretrained LLM with a systematic evaluator and evolves *programs* rather than solutions, yielding new results in extremal combinatorics and bin packing; AlphaEvolve [3] generalises this to an evolutionary coding agent over a program database grounded by automatic evaluation. The Gödel-machine ideal [5]—provably beneficial self-modification—is relaxed by the Darwin Gödel Machine [4], which empirically validates each self-modification and grows an archive of agents. Dream-RSI [1] (Zheng et al., Google/DeepMind/UMD, arXiv:2609.14858) is the method this library implements; the official repository is <https://github.com/zhengkid/Dream-RSI>. Our contribution is at the library/infrastructure level within this family: the mechanisms above are proven at

scale by their authors; we ship the same loop pattern as a dependency-free, budget-guarded, OpenAI-compatible tool and add the curator and the exploration-isolating benchmark.

Memory and self-generated policies in agents. Reflexion [6] shows that verbal self-feedback stored in an episodic memory materially improves subsequent attempts—the same principle our knowledge curator operationalises persistently across runs; Voyager [7] builds a persistent skill library of verified programs, the closest ancestor of our replay-gated policy promotion.

Independent implementations. Since the paper’s release several independent implementations have appeared: a model-agnostic adapter SDK with strict replay and evidence-based promotion gates [10], a local-Qwen CUDA-kernel exploration on DGX Spark [11], and agent/skill variants [12]. Against peers, this library differentiates on: the always-on budget-guarded supervisor; LLM-written policy code (as opposed to a fixed policy menu) with counterfactual-replay promotion; a dedicated exploration benchmark (**bench-policy**); a curated lesson knowledge base with retrieval feedback; persistent cross-run memory; a live dashboard; zero dependencies; MIT license.

A different layer. OpenRSI/OpenMLE/Frontis-MA1 [8, 9] post-train weights (SFT+RL, a 35B meta-evolution agent on MLE-Bench); the two layers are complementary—their trained improver could be the frozen model behind our client, our dreaming loop could sit on top of their search.

8 Limitations

We state plainly what the numbers do and do not show.

1. **Scripted worlds, not model quality.** Every headline number comes from deterministic benchmarks with a scripted solver; the trap scores are fixed by construction, the scripted solver treats a stored lesson text as a genuine insight when it is visible in the prompt, and the “policy” returned by the scripted model is a canned program. The benchmarks therefore validate the *machinery*—sandboxes, gates, promotion, retrieval, budget—and prove the arms differ *causally* through memory, not that any live model writes good lessons or policies. Live-model evaluation is the required next experiment.
2. **Benchmark self-design risk.** The decoy-trap suite was designed by the same author as the loop it measures. Mitigations: family scores fixed and asserted by construction; arm contracts enforced by tests (greedy must fail, arms must differ causally); determinism lets anyone re-measure with one command; five seed sets reported.
3. **Single-node sandbox.** `python -I` with scrubbed environment and timeout is a real boundary against casual escape but is not a container; deployments with hostile task sources should add OS-level isolation.
4. **Scope.** No weight updates, no multi-agent dynamics, no adversarial red-teaming of promoted policies; the knowledge base is per-category text, not a structured world model.

9 Conclusion

Open Dream-RSI turns the Dream-RSI method into a deployable, budget-safe, dependency-free library and adds the two mechanisms a running loop actually needs to compound—evidence-gated learned policies and a curated, self-pruning knowledge base—each backed by a deterministic benchmark that isolates exactly the capability it claims. At library scale: 54% fewer API

calls at equal solve quality; on a suite where greedy exploration solves nothing, replay-gated policies reach 77% and remembered knowledge 72% cumulative (100% by cycle 5–8) at $\sim 4\times$ better cost-per-solve than random escape. All artefacts pass structural gates before touching the loop; the gate, not trust, decides.

References

- [1] Zheng et al., *Dream-RSI: Recursive Self-Improvement through Evolving Worlds*, arXiv:2609.14858, 2026.
- [2] B. Romera-Paredes, M. Barekatin, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, A. Fawzi, *Mathematical discoveries from program search with large language models*, Nature 625(7995), 2024. DOI: 10.1038/s41586-023-06924-6.
- [3] A. Novikov, N. Vĩ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, et al., *AlphaEvolve: A coding agent for scientific and algorithmic discovery*, arXiv:2506.13131, 2025.
- [4] J. Zhang, S. Hu, C. Lu, R. Lange, J. Clune, *Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents*, arXiv:2505.22954, 2025.
- [5] J. Schmidhuber, *Gödel Machines: Fully Self-Referential Optimal Universal Self-Improvers*, in B. Goertzel and C. Pennachin (eds.), *Artificial General Intelligence*, Springer, 2006.
- [6] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao, *Reflexion: Language Agents with Verbal Reinforcement Learning*, Advances in Neural Information Processing Systems 36, 2023. arXiv:2303.11366.
- [7] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, A. Anandkumar, *Voyager: An Open-Ended Embodied Agent with Large Language Models*, arXiv:2305.16291, 2023.
- [8] FrontisAI, *OpenRSI / OpenMLE / Frontis-MA1*, <https://github.com/FrontisAI/OpenRSI>.
- [9] J. S. Chan, N. Chowdhury, et al., *MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering*, arXiv:2410.07095, 2024.
- [10] TheAstrayDev, *dream-rsi-sdk*, <https://github.com/TheAstrayDev/dream-rsi-sdk>.
- [11] robinber, *dream-rsi-spark*, <https://github.com/robinber/dream-rsi-spark>.
- [12] lesterppo, *hermes-dream-rsi*, <https://github.com/lesterppo/hermes-dream-rsi>; Harkit2004, ... *dream-rsi-skill*; mailbobg, *Pi-RSI*.